Informe Evaluación Parcial II

Sección: 004V

Integrantes:

Daniel Ignacio Betancourt Jerez
Rodrigo Del Tránsito Muñoz Catalán
Moisés Alonso Salinas Castro

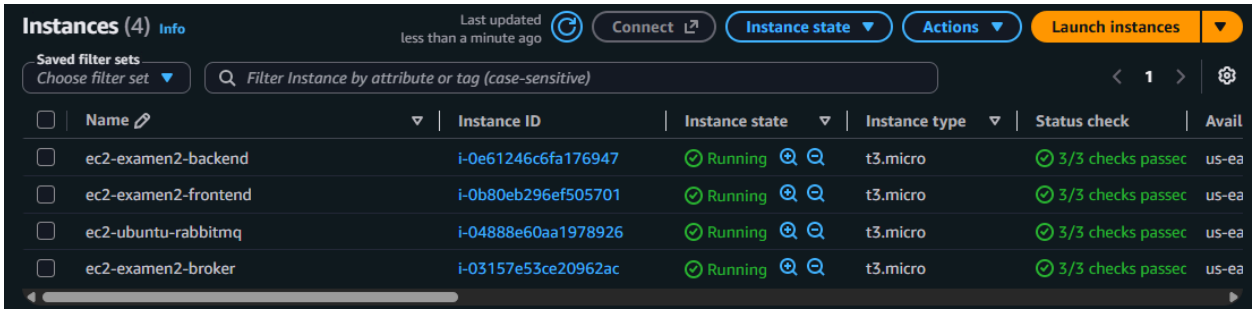
Profesor:

Francisco Javier Briones Lavados

Fecha:

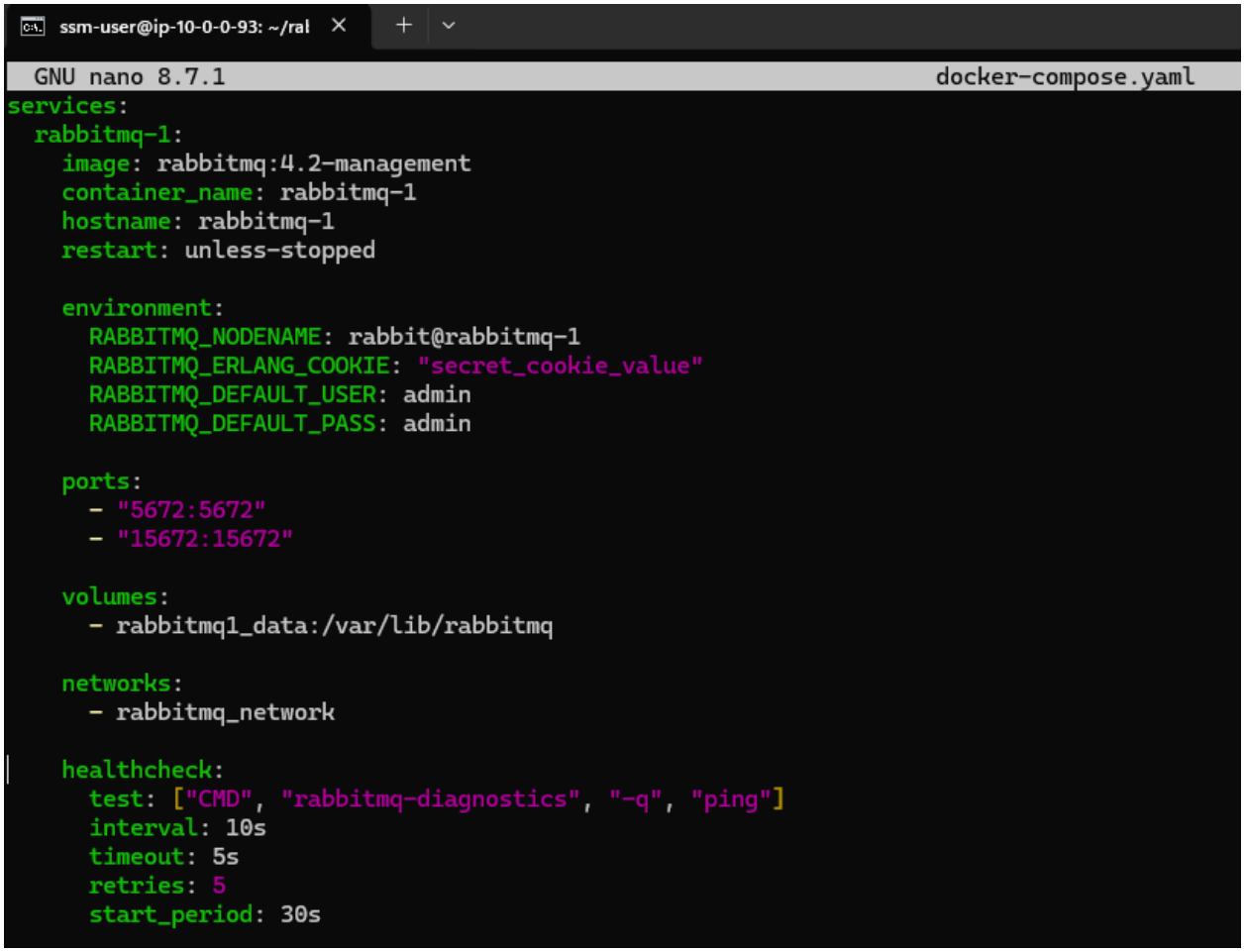
26/10/2026

Despliegue de ec2 (frontend - backend - broker)



Se desplegaron instancias EC2 en AWS para separar los componentes de frontend, backend y broker de mensajería. La consola evidencia las instancias en ejecución y sus verificaciones de estado satisfactorias.

EC2-BROKER


ssm-user@ip-10-0-0-93: ~/ral
GNU nano 8.7.1
docker-compose.yaml
services:
rabbitmq-1:
image: rabbitmq:4.2-management
container_name: rabbitmq-1
hostname: rabbitmq-1
restart: unless-stopped
environment:
RABBITMQ_NODENAME: rabbit@rabbitmq-1
RABBITMQ_ERLANG_COOKIE: "secret_cookie_value"
RABBITMQ_DEFAULT_USER: admin
RABBITMQ_DEFAULT_PASS: admin
ports:
- "5672:5672"
- "15672:15672"
volumes:
- rabbitmq1_data:/var/lib/rabbitmq
networks:
- rabbitmq_network
healthcheck:
test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
interval: 10s
timeout: 5s
retries: 5
start_period: 30s

```yaml
  rabbitmq-2:
    image: rabbitmq:4.2-management
    container_name: rabbitmq-2
    hostname: rabbitmq-2
    restart: unless-stopped

    environment:
      RABBITMQ_NODENAME: rabbit@rabbitmq-2
      RABBITMQ_ERLANG_COOKIE: "secret_cookie_value"
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin

    ports:
      - "5673:5672"
      - "15673:15672"

    volumes:
      - rabbitmq2_data:/var/lib/rabbitmq

    networks:
      - rabbitmq_network

    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
```

```yaml
networks:
  rabbitmq_network:
    driver: bridge

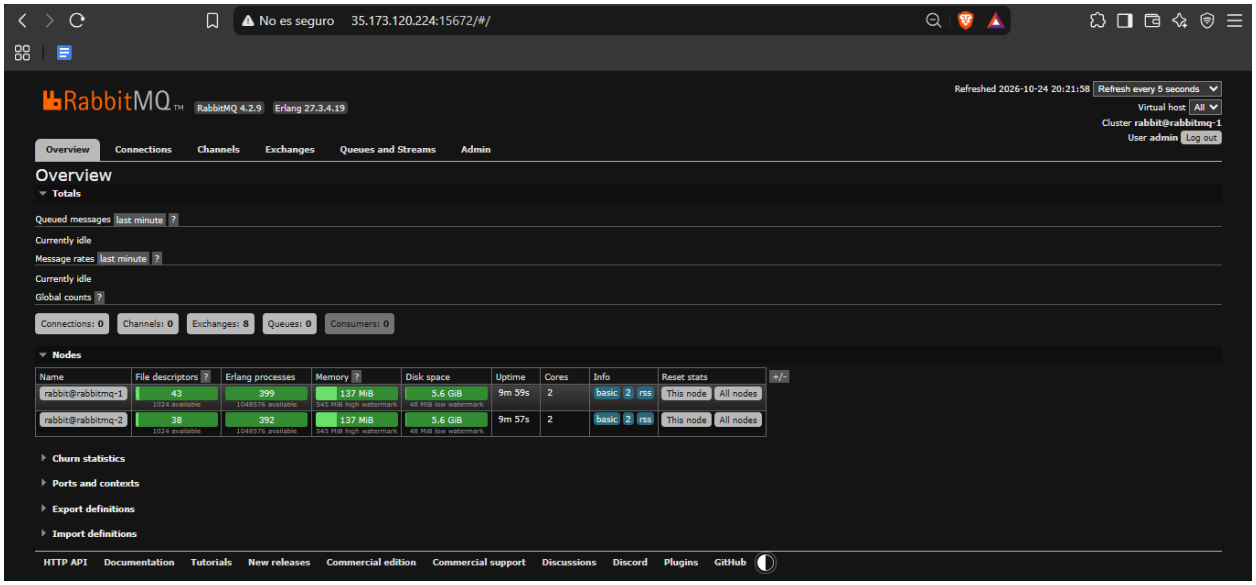
volumes:
  rabbitmq1_data:
  rabbitmq2_data:
```

Se configuró el primer nodo RabbitMQ mediante Docker Compose, definiendo la imagen, los puertos de comunicación, el almacenamiento persistente y las verificaciones de salud para supervisar el funcionamiento del contenedor.

```
ssm-user@ip-10-0-0-93:~/rabbit-cluster$ sudo docker compose up -d
[+] Running 3/3
 ✔Network rabbit-cluster_rabbitmq_network  Created                                                                    0.1s
 ✔Container rabbitmq-2                    Started                                                                    0.5s
 ✔Container rabbitmq-1                    Started                                                                    0.5s
ssm-user@ip-10-0-0-93:~/rabbit-cluster$ sudo docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED          STATUS                    PORTS
                                                                                                                    NAMES
3f61d7e02f72   rabbitmq:4.2-management   "docker-entrypoint.s…"   23 seconds ago   Up 22 seconds (healthy)   4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/
tcp, [::]:5672->5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp   rabbitmq-1
ec61f6d2ed49   rabbitmq:4.2-management   "docker-entrypoint.s…"   23 seconds ago   Up 22 seconds (healthy)   4369/tcp, 5671/tcp, 15671/tcp, 15691-15
692/tcp, 25672/tcp, 0.0.0.0:5673->5672/tcp, [::]:5673->5672/tcp, 0.0.0.0:15673->15672/tcp, [::]:15673->15672/tcp   rabbitmq-2
ssm-user@ip-10-0-0-93:~/rabbit-cluster$ sudo docker exec rabbitmq-1 rabbitmq-diagnostics -q ping
Ping succeeded
ssm-user@ip-10-0-0-93:~/rabbit-cluster$  sudo docker exec rabbitmq-2 rabbitmqctl join_cluster rabbit@rabbitmq-1
Clustering node rabbit@rabbitmq-2 with rabbit@rabbitmq-1
```

Se incorporó un segundo nodo RabbitMQ con puertos y volumen independientes, utilizando una red Docker compartida para permitir la comunicación entre ambos contenedores.

Se ejecutó Docker Compose para iniciar ambos nodos y posteriormente se utilizó el comando rabbitmqctl join_cluster para integrar el segundo nodo al clúster de mensajería.



Se verificó el estado del clúster desde RabbitMQ Management, observando los nodos disponibles y sus indicadores operativos. Esta interfaz permite supervisar el funcionamiento del broker.

EC2 BACKEND



Se organizó el backend en Python, separando la API Flask, la configuración de RabbitMQ, los servicios de mensajería, los consumidores y los archivos necesarios para el despliegue con Docker.

```
ssm-user@ip-10-0-0-101: ~/ba
ssm-user@ip-10-0-0-101:~/backend$ sudo docker compose up -d --build
WARN[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 1.7s (34/43)                                                
```

```
 => => naming to docker.io/library/backend-worker-prioritario:latest                                                0.0s
 => => unpacking to docker.io/library/backend-worker-prioritario:latest                                             0.0s
 => [flask-api] exporting to image                                                                                  0.4s
 => => exporting layers                                                                                             0.0s
 => => exporting manifest sha256:02fb002933bad12a65750b7ac2b34eb372168123f9723728 43e269b7207bd1ba                   0.0s
 => => exporting config sha256:cbc914b3b5563ee9dc4e14fafcd9e6a03fa6107f22ca9701526b1faf229f2a04                     0.1s
 => => exporting attestation manifest sha256:2826fb4124b8db5b7de84ea0f0b6ae801cb690b9b1d44af73eee31beeb34ba76       0.1s
 => => exporting manifest list sha256:84844a8121e3af997dd5b60d9cf3882f0ebd594be3da6d07dc37de54248c9719              0.1s
 => => naming to docker.io/library/backend-flask-api:latest                                                         0.0s
 => => unpacking to docker.io/library/backend-flask-api:latest                                                      0.0s
 => [worker-normal] exporting to image                                                                              0.4s
 => => exporting layers                                                                                             0.0s
 => => exporting manifest sha256:c674e8683a1a7a5de16974d6bd7e0b83a678004e57a5e66076fb1fa265fa46ed                   0.1s
 => => exporting config sha256:51a3535d6cd4bd6f4bb783870bdae0fc141d55ffd6921336a8af3a11fb5d8279                     0.1s
 => => exporting attestation manifest sha256:ea8dd31208faa101b8df878fd96ab4744bba78929356680d17ed4819eb57da42       0.1s
 => => exporting manifest list sha256:afedbe2cbd033c93ca178d8c01498e333ce548481ffb5e1516912b9798424715              0.1s
 => => naming to docker.io/library/backend-worker-normal:latest                                                     0.0s
 => => unpacking to docker.io/library/backend-worker-normal:latest                                                  0.0s
 => [rabbitmq-admin] resolving provenance for metadata file                                                         0.1s
 => [flask-api] resolving provenance for metadata file                                                              0.1s
 => [worker-prioritario] resolving provenance for metadata file                                                     0.1s
 => [worker-normal] resolving provenance for metadata file                                                          0.0s
[+] Running 10/10
 ✔flask-api                      Built                                                                             0.0s
 ✔rabbitmq-admin                 Built                                                                             0.0s
 ✔rabbitmq-init                  Built                                                                             0.0s
 ✔worker-normal                  Built                                                                             0.0s
 ✔worker-prioritario             Built                                                                             0.0s
 ✔Container backend-rabbitmq-init-1  Exited                                                                        1.0s
 ✔Container worker-normal        Started                                                                           1.5s
 ✔Container worker-prioritario   Started                                                                           1.5s
 ✔Container flask-api            Started                                                                           1.3s
 ✔Container rabbitmq-admin       Started                                                                           1.4s
```

Se ejecutó docker compose up -d --build para construir las imágenes del backend e instalar las dependencias necesarias, preparando los servicios para su ejecución en contenedores.

Docker completó la construcción de las imágenes correspondientes a Flask, administración de RabbitMQ y consumidores de mensajes normales y prioritarios.

Docker completó la construcción de las imágenes correspondientes a Flask, administración de RabbitMQ y consumidores de mensajes normales y prioritarios.

Se iniciaron los servicios del backend mediante Docker Compose. El contenedor de inicialización finalizó correctamente y los servicios permanentes quedaron en ejecución.

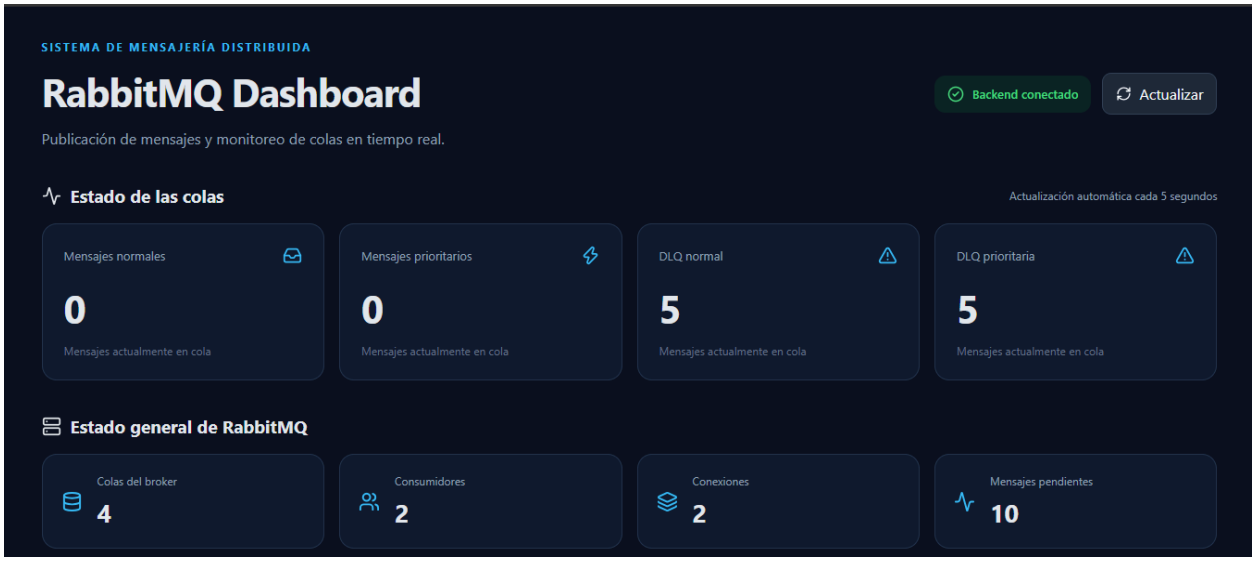
```
ssm-user@ip-10-0-0-101:~/backend$ sudo docker compose ps -a
NAME                      IMAGE                       COMMAND                  SERVICE             CREATED          STATUS                      POR
TS
backend-rabbitmq-init-1   backend-rabbitmq-init       "python rabbitmq.py"     rabbitmq-init       11 seconds ago   Exited (0) 9 seconds ago
flask-api                 backend-flask-api           "gunicorn --bind 0.0…"   flask-api           10 seconds ago   Up 9 seconds                0.0
.0.0:5000->5000/tcp, [::]:5000->5000/tcp, 5001/tcp
rabbitmq-admin            backend-rabbitmq-admin      "gunicorn --bind 0.0…"   rabbitmq-admin      10 seconds ago   Up 9 seconds                500
0/tcp, 127.0.0.1:5001->5001/tcp
worker-normal             backend-worker-normal       "python worker.py"       worker-normal       10 seconds ago   Up 9 seconds                500
0-5001/tcp
worker-prioritario        backend-worker-prioritario  "python worker.py"       worker-prioritario  10 seconds ago   Up 9 seconds                500
0-5001/tcp
```

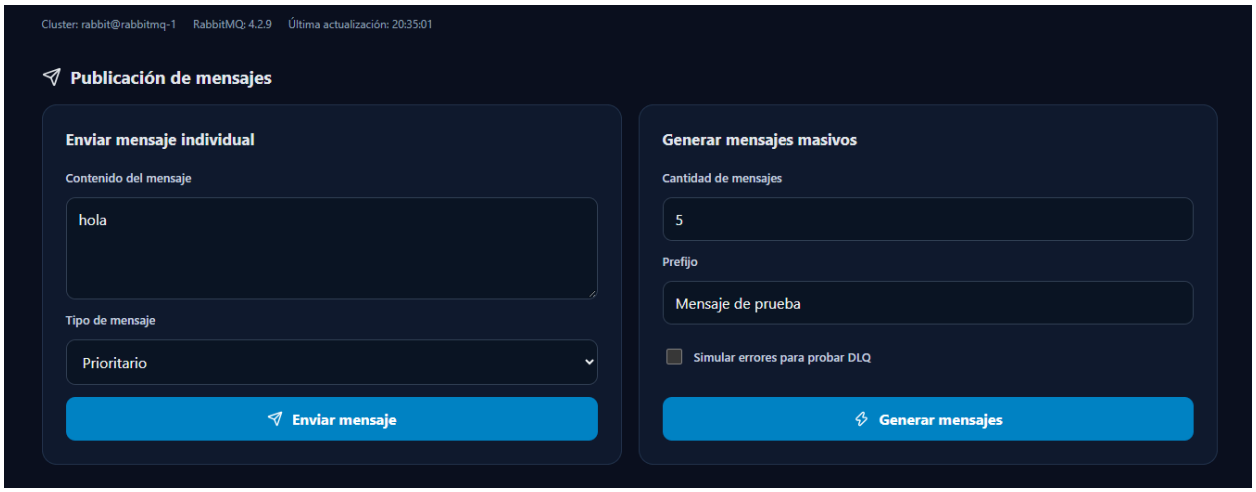
Mediante docker compose ps -a se comprobó el estado de los contenedores, verificando la ejecución de Flask, el servicio administrativo y los dos consumidores de mensajes.

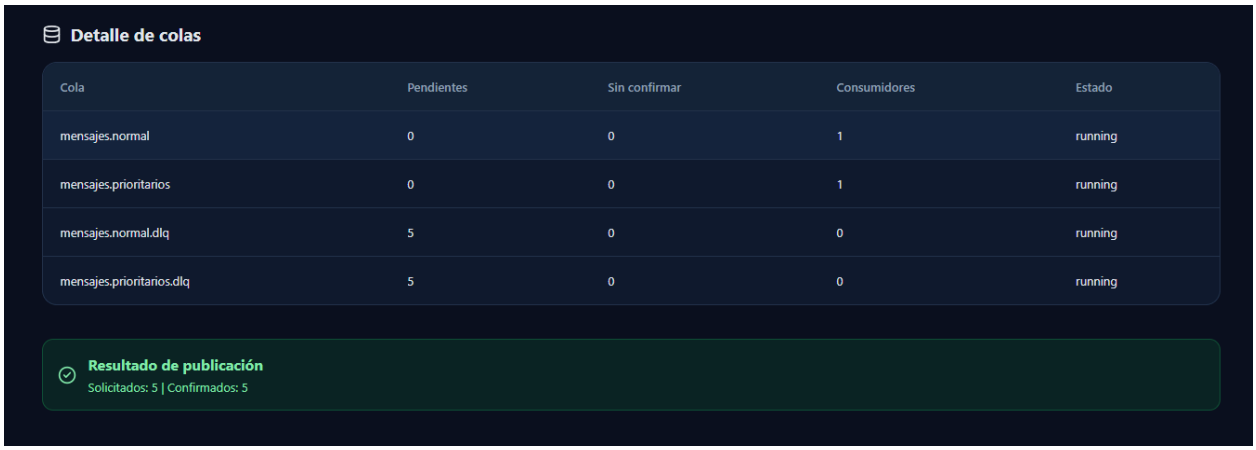
EC2 FRONTEND

```
ssm-user@ip-10-0-0-126:~/rabbitmqfront$ ls -la
total 128
drwxr-xr-x   5 ssm-user ssm-user  4096 Oct 25 22:31 .
drwxr-x---   6 ssm-user ssm-user  4096 Oct 25 23:13 ..
-rw-r--r--   1 ssm-user ssm-user    39 Oct 25 22:31 .env
-rw-r--r--   1 ssm-user ssm-user   253 Oct 25 22:06 .gitignore
-rw-r--r--   1 ssm-user ssm-user  1027 Oct 25 22:06 README.md
-rw-r--r--   1 ssm-user ssm-user   568 Oct 25 22:06 eslint.config.js
-rw-r--r--   1 ssm-user ssm-user   365 Oct 25 22:06 index.html
drwxr-xr-x 113 ssm-user ssm-user  4096 Oct 25 23:30 node_modules
-rw-r--r--   1 ssm-user ssm-user 80493 Oct 25 23:30 package-lock.json
-rw-r--r--   1 ssm-user ssm-user   641 Oct 25 22:15 package.json
drwxr-xr-x   2 ssm-user ssm-user  4096 Oct 25 22:06 public
drwxr-xr-x   4 ssm-user ssm-user  4096 Oct 25 23:30 src
-rw-r--r--   1 ssm-user ssm-user   214 Oct 25 22:24 vite.config.js
ssm-user@ip-10-0-0-126:~/rabbitmqfront$
```

Se desarrolló el frontend utilizando React y Vite, organizando los componentes, estilos y servicios de conexión con la API Flask para permitir la interacción con RabbitMQ.





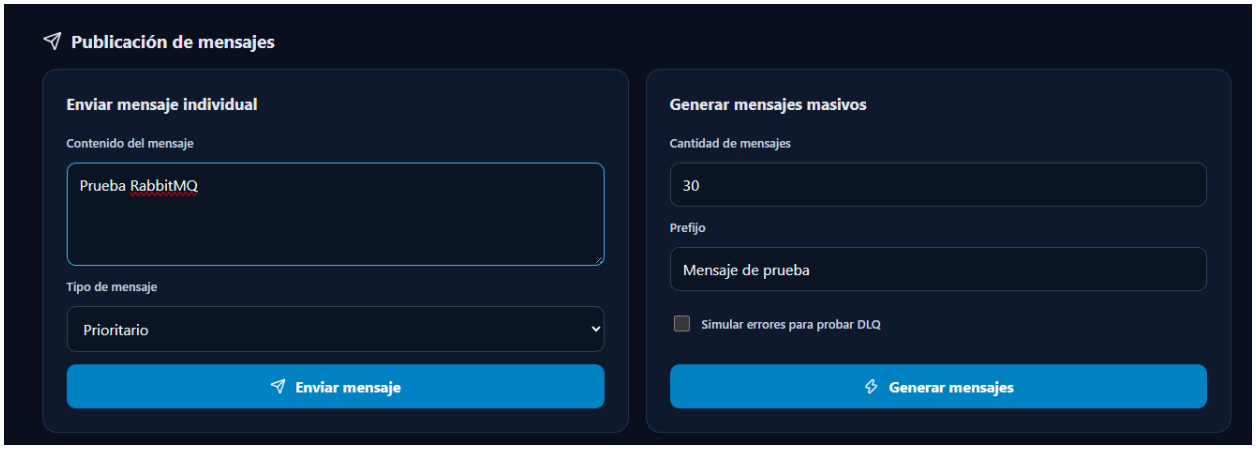


Se implementó un dashboard que muestra el estado de las colas normales, prioritarias y de mensajes fallidos (DLQ), junto con información general del broker y actualización automática de las métricas.

Se incorporaron formularios para enviar mensajes individuales y generar lotes de mensajes, permitiendo seleccionar su prioridad y activar la simulación de errores para probar las colas DLQ.

El dashboard presenta el estado de cada cola, sus consumidores y los mensajes pendientes. Además, se observa la confirmación de publicación de cinco mensajes y la presencia de mensajes en las colas DLQ.

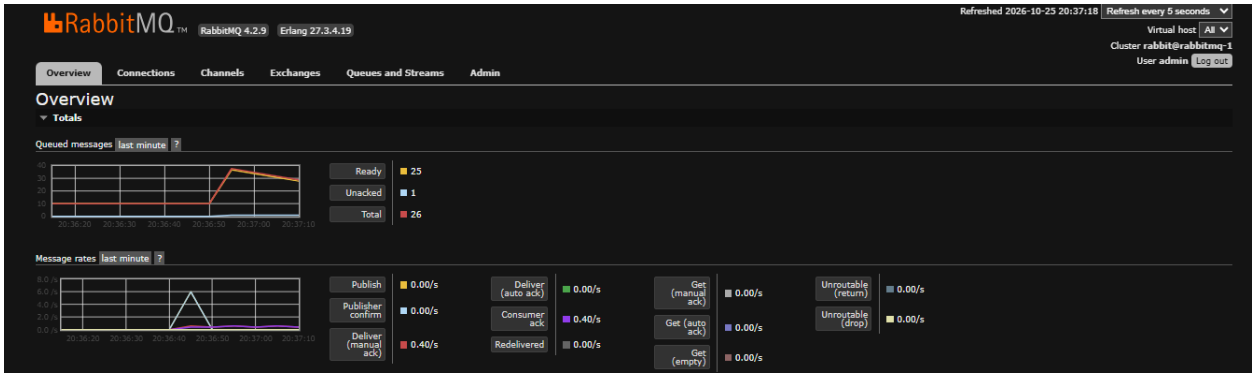
Pruebas

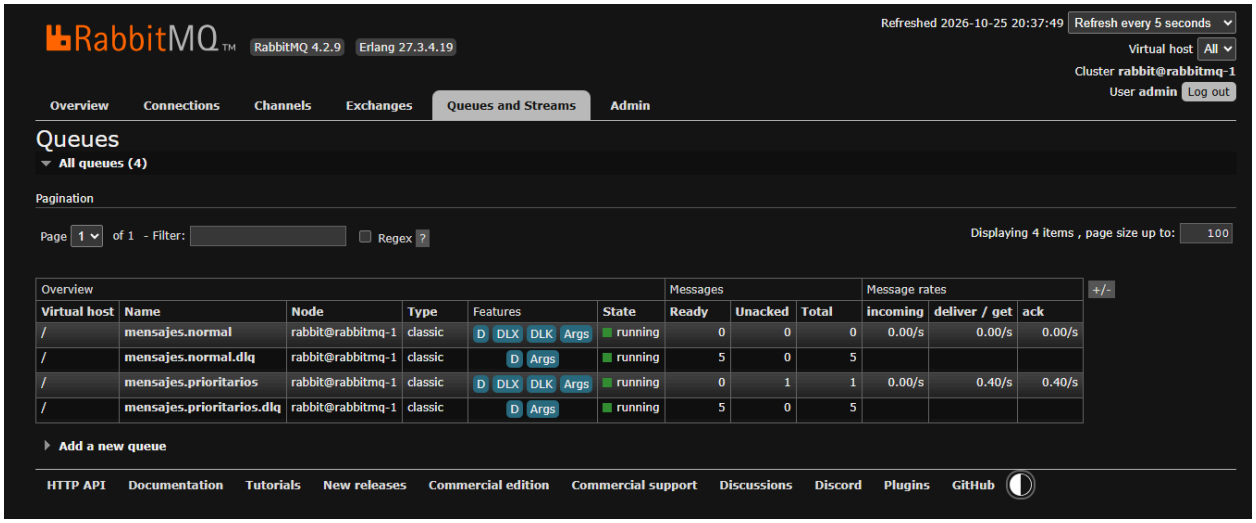


Se realizó una prueba de generación de 30 mensajes mediante el frontend, utilizando la prioridad seleccionada para verificar la comunicación con el backend y la publicación hacia RabbitMQ.

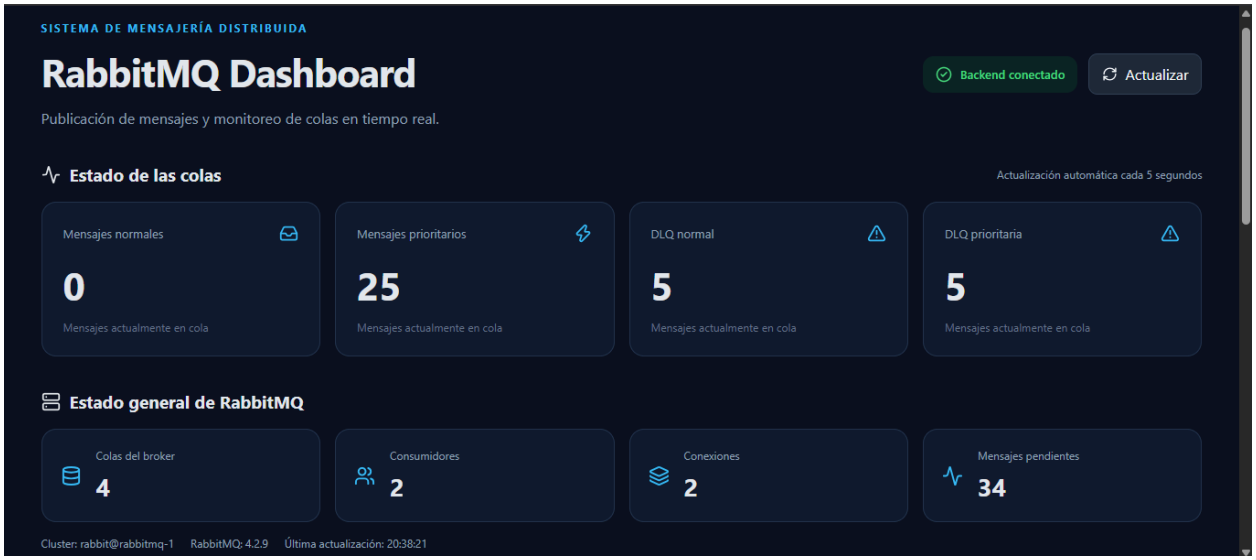


La interfaz confirmó la publicación de los 30 mensajes solicitados. Posteriormente, RabbitMQ Management permitió observar la actividad de mensajería mediante sus indicadores y gráficos de monitoreo.





Desde RabbitMQ Management se verificó la existencia de las cuatro colas configuradas, incluyendo las colas normales, prioritarias y sus respectivas DLQ, junto con sus cantidades de mensajes.



El dashboard reflejó los mensajes acumulados en las colas, mostrando 25 mensajes prioritarios y cinco mensajes en cada DLQ. Esto evidencia la consulta de métricas desde RabbitMQ hacia el frontend.

Conclusión

La implementación permitió integrar un frontend React, una API Flask y un broker RabbitMQ desplegados en AWS EC2. Las pruebas evidenciaron la publicación de mensajes normales y prioritarios, el funcionamiento de las colas de mensajes fallidos (DLQ) y la visualización de métricas desde el dashboard. Con ello, se comprobó la comunicación entre los componentes y el funcionamiento del sistema de mensajería distribuida.